

Onboard API

Reference for the desktop client

START HERE

Overview

Authentication

Data model

RESOURCES

Companies

Programs

Employees

PUBLIC ENDPOINTS

Employee portal (RPC)

Public share (RPC)

OTHER

File storage

Security model

Realtime (optional)

Onboard API reference

The backend is Supabase (Postgres + Auth + Storage) — there is no separate REST server to build against. Every endpoint below is called through a Supabase client SDK. This doc lists what tables and functions exist, what each one expects, and which ones require a signed-in user versus none at all.

Overview

Ask the web team for the two values every client needs — they're the same for web, iOS, Android, and desktop:

```
SUPABASE_URL      = https://<project-ref>.supabase.co
SUPABASE_ANON_KEY = <anon / publishable key>
```

Pick the SDK for your platform — all of them expose the same three surfaces used below:

`.auth` , `.from(table)` , `.rpc(fn)` , `.storage` .

PLATFORM	PACKAGE
Swift / macOS / iOS	supabase-swift
Kotlin / Android / desktop (JVM)	supabase-kt
.NET	supabase-csharp
Electron / Node	@supabase/supabase-js
Flutter	supabase_flutter

No API keys or secrets are baked into any client — the anon key is safe to ship in a compiled app. Every access rule below is enforced server-side by Postgres row-level security, not by the client.

Authentication

Email + password, handled entirely by Supabase Auth. There is no custom login endpoint.

AUTH `signInWithPassword({ email, password })`

Returns a session (access + refresh token), persisted by the SDK automatically. Do this on app launch too — call `auth.getSession()` first to restore an existing session before showing a login screen.

AUTH `signUp({ email, password, options: { data: { name } } })`

Creates the user and an empty `profiles` row (via DB trigger, role defaults to `admin`, no company yet). Your client must then insert a `companies` row and update `profiles.company_id` — see [Companies](#).

AUTH `signOut()`

Clears the local session. Subscribe to `auth.onAuthStateChange()` to react to expiry across the app.

AUTH `from('profiles').select('*').eq('id', userId).single()`

After sign-in, fetch the profile to get `role` (`admin` or `owner`) and `company_id` — this determines what the rest of the app can show.

Data model

Six tables, one company per tenant. Every row below the company level carries its own `company_id` so access rules never need a join.

TABLE	KEY COLUMNS	NOTES
<code>companies</code>	<code>name, industry, departments[]</code>	One row per tenant
<code>profiles</code>	<code>id = auth user id, role, company_id</code>	<code>role: admin owner</code>
<code>programs</code>	<code>name, status, share_token</code>	<code>status: draft published</code>
<code>stages</code>	<code>program_id, order, deadline</code>	Ordered steps in a program
<code>materials</code>	<code>stage_id, type, data (jsonb)</code>	<code>type: video reading checklist quiz task document meeting</code>
<code>employees</code>	<code>assigned_program_id, access_token</code>	<code>access_token</code> powers the public portal link
<code>employee_progress</code>	<code>employee_id, material_id, detail</code>	Join table — completion + submitted data

Companies company profile & departments

SELECT `from('companies').select('*').eq('id', companyId).single()`

signed in

UPDATE

`from('companies').update({ name, logo_url, primary_color, departments }).eq('id', companyId)`

signed in · admin

`departments` is a plain `text[]` — read, append/rename/remove client-side, write the whole array back.

Programs, stages & materials onboarding content

SELECT

`from('programs').select('*', stages(*, materials(*))).eq('company_id', companyId)`

signed in

One nested query returns the full program tree. `materials.data` is jsonb holding type-specific fields (quiz questions, checklist items, video url, etc) — merge it onto the row client-side.

INSERT

`from('programs').insert({ company_id, name, status: 'draft' }).select().single()`

signed in

UPDATE `from('stages').update({ name, deadline, order }).eq('id', stageId)`

signed in

DELETE `from('materials').delete().eq('id', materialId)`

signed in

UPDATE `from('programs').update({ share_token: uuid }).eq('id', programId)`

signed in

Setting a `share_token` turns on the public read-only dashboard for that program (see [Public share](#)). Set it back to `null` to revoke.

Employees roster & progress

SELECT

```
from('employees').select('*', employee_progress(material_id, detail)).eq('company_id', companyId)
```

signed in

INSERT

```
from('employees').insert({ company_id, name, email, department, employment_type, manager }).select().single()
```

signed in

`access_token` is generated by the database automatically (default `gen_random_uuid()`) — read it back from the insert response, don't set it yourself.

UPDATE

```
from('employees').update({ assigned_program_id }).eq('id', employeeId)
```

signed in

Assigning a new program should also clear old progress:

```
from('employee_progress').delete().eq('employee_id', employeeId) .
```

DELETE

```
from('employees').delete().eq('id', employeeId)
```

signed in

Employee portal rpc · no login

Employees never sign in. Each one gets a link containing their `access_token` — the desktop app (or a kiosk view) calls these three functions directly with the token from the URL/deep-link. Each function validates the token server-side before touching any data.

RPC `rpc('get_employee_portal', { p_token })` public

Returns one jsonb blob: `{ employee, company, program, completed }`. `program` already includes nested stages + materials.

```
{
  "employee": { "id": "...", "name": "Sari Wulandari", ... },
  "company": { "name": "Sunrise Bistro", "primary_color": "#f59e0b", ... },
  "program": { "name": "...", "stages": [ { "materials": [ ... ] } ] },
  "completed": [ { "material_id": "...", "detail": null } ]
}
```

RPC `rpc('mark_material_complete', { p_token, p_material_id })` public

RPC `rpc('record_material_detail', { p_token, p_material_id, p_detail })` public

`p_detail` is freeform jsonb — quiz answers, checklist state, signature name, whatever the material type needs.

Public share rpc · no login

RPC `rpc('get_shared_program', { p_token })` public

Read-only summary for a program's `share_token` — stage list, material counts, and aggregate stats (`enrolled`, `completed`, `avg_pct`). Deliberately excludes employee names — this is meant to be shown on a public link.

File storage logos & banners

UPLOAD

```
storage.from('company-logos').upload(`${companyId}/logo.png`, file, { upsert: true })
```

signed in

READ

```
storage.from('company-logos').getPublicUrl(path)
```

public

Both buckets (`company-logos` , `program-images`) are public-read. Upload path must start with your own `company_id` — storage policy rejects any other prefix.

Security model

Nothing above needs client-side permission checks — every table has row-level security keyed off the signed-in user's `company_id`:

ROLE	CAN SEE	CAN WRITE
admin	Own company's rows only	Own company's rows only
owner	All companies (read-only)	Nothing outside their own company
anon (no session)	Only via the RPC functions above	Only via the RPC functions above

Building an owner/admin console? Just query tables directly with a signed-in owner session — `from('companies').select(*)` returns every tenant automatically, no special flag needed.

Realtime optional

Not used by the web app yet, but every table already has RLS, so Supabase Realtime works out of the box if the desktop app wants live updates (e.g. a kiosk dashboard that updates as employees complete steps):

```
supabase
  .channel('employee-progress')
  .on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'employee_progress' }, () => {
    // ...
  })
  .subscribe()
```